Instituto Politécnico Nacional

Centro de Investigación en Computación

Diseño y analisis de algoritmos

Practica 1: Emparejamiento Estable

Alumno:
Ronquillo Núñez Braulio Alberto

Profesor:
Raul Acosta Bermejo

27 de febrero de 2026

1. Introducción

El Problema de Emparejamiento Estable (Stable Matching Problem) fue formalizado por Gale y Shapley en 1962 [1]. El problema consiste en emparejar dos conjuntos disjuntos de igual tamaño, donde cada participante posee un orden completo de preferencias sobre los elementos del conjunto opuesto. El objetivo es encontrar un emparejamiento que sea estable.

Un emparejamiento es estable si no existe un par bloqueante, es decir, un par de participantes que prefieran estar emparejados entre sí antes que con sus respectivas parejas asignadas. La estabilidad garantiza que ningún par tenga incentivos para desviarse del resultado obtenido.

El algoritmo de Aceptación Diferida propuesto por Gale y Shapley garantiza la existencia de al menos un emparejamiento estable para cualquier instancia válida del problema [1]. Desde el punto de vista computacional, este algoritmo tiene complejidad temporal $O(N^2)$ en el peor caso [2], ya que cada participante puede realizar a lo sumo N propuestas.

El problema tiene aplicaciones relevantes en diseño de mecanismos y asignación de recursos, como en sistemas de residencias médicas y mercados laborales [3]. En esta práctica se implementa el algoritmo de Gale–Shapley considerando ambas variantes posibles (cuando propone cada conjunto) y se analiza experimentalmente su comportamiento temporal.

1.1. Objetivos

El objetivo de esta práctica es comprobar experimentalmente el comportamiento asintótico del algoritmo de Gale y Shapley que encuentra el emparejamiento estable de dos conjuntos de hombres y mujeres cuya cardinalidad de ambos es N. Es decir, resolver el SMP (Stable Matching Problem) en su versión básica

2. Metodologia

La metodología empleada en esta práctica consistió en implementar el algoritmo de Gale–Shapley para resolver el Problema de Emparejamiento Estable y posteriormente evaluar experimentalmente su comportamiento temporal.

En una primera etapa se desarrolló una implementación eficiente del algoritmo considerando ambas variantes posibles: cuando el conjunto de hombres realiza las propuestas y cuando el conjunto de mujeres lo hace. Para optimizar el desempeño, se utilizó un esquema de *ranking inverso* que permite comparar preferencias en tiempo constante $O(1)$, evitando recorridos lineales en las listas de preferencias.

En una segunda etapa se generaron automáticamente múltiples instancias de entrada. Cada instancia contiene dos conjuntos de tamaño N con listas completas de preferencias generadas aleatoriamente. Los valores de N fueron seleccionados de forma creciente para observar el comportamiento asintótico esperado. La generación de datos se automatizó mediante un script independiente, lo que permitió controlar sistemáticamente el tamaño de las entradas.

Posteriormente, para cada instancia generada, se ejecutó el algoritmo y se midieron los tiempos de ejecución. Siguiendo las indicaciones del enunciado, se diferenciaron distintas secciones del código para evaluar su impacto en el tiempo total:

- Tiempo de lectura de datos.
- Tiempo de ejecución del algoritmo (emparejamiento).
- Tiempo de impresión de resultados.

Las mediciones se realizaron utilizando un temporizador de alta resolución (`perf_counter`) y los resultados fueron almacenados en una tabla estructurada que relaciona el tamaño N con el tiempo correspondiente.

Finalmente, los tiempos obtenidos fueron graficados utilizando tanto escala lineal como escala logarítmica para el eje de N, con el objetivo de analizar la forma del crecimiento temporal y contrastarlo con la complejidad teórica $O(N^2)$.

2.1. Herramientas

Para el desarrollo de la práctica se utilizaron las siguientes herramientas:

- Lenguaje de programación Python 3 para la implementación del algoritmo y generación de instancias.
- Bibliotecas estándar de Python para medición de tiempos y manejo de estructuras de datos.
- Biblioteca `matplotlib` para la generación de gráficas.
- Entorno de desarrollo local y sistema de control de versiones Git para organización del código.

3. Resultados

Para evaluar el comportamiento temporal del algoritmo de Gale–Shapley se generaron múltiples instancias de entrada con tamaños crecientes N. Para ca-

da instancia se midió el tiempo necesario para ejecutar únicamente el proceso de emparejamiento, diferenciándolo del tiempo de lectura e impresión.

La Tabla 1 muestra los tiempos obtenidos para distintos valores de N.

Tabla 1: Tiempos de ejecución del algoritmo de Gale–Shapley

N	Tiempo (ms)
32	0.255
64	0.694
128	2.317
256	10.178
384	24.806
512	41.967
640	62.640
768	89.410
896	119.286
1024	215.997
1200	220.035
1400	332.430
1600	405.424
1800	532.750
1900	586.778
2000	646.559

Con base en estos resultados se generaron gráficas utilizando escala lineal y escala logarítmica para el eje de N, tal como se recomienda en el enunciado de la práctica. La gráfica en escala lineal permite observar el crecimiento acelerado del tiempo conforme aumenta el tamaño de entrada, mientras que la gráfica en escala logarítmica facilita identificar la tendencia asintótica del algoritmo.

Los datos experimentales muestran un crecimiento no lineal del tiempo de ejecución, consistente con el comportamiento cuadrático esperado teóricamente para el algoritmo de Gale–Shapley.

4. Análisis de resultados

La Figura 1 muestra el tiempo de ejecución del algoritmo en función del tamaño de entrada N en escala lineal. Se observa que el crecimiento no es lineal, sino que aumenta de manera acelerada conforme N se incrementa. Este comportamiento es consistente con una complejidad mayor que $O(N)$.

Para analizar el comportamiento asintótico con mayor claridad, se utilizó una representación log-log, mostrada en la Figura 2. En esta escala, si el comportamiento sigue una ley polinómica $T(N) = cN^k$, los datos deben aproximarse a una recta cuya pendiente corresponde al exponente k.

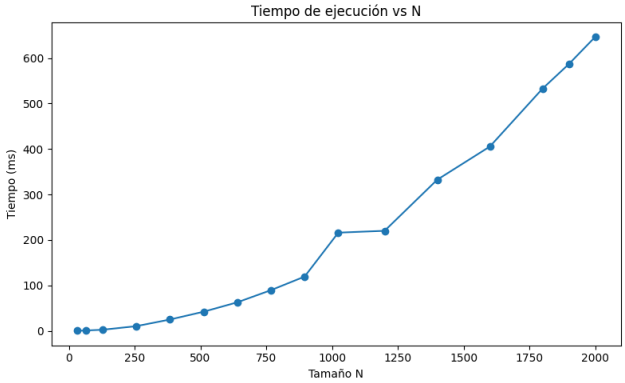

Figura 1: Tiempo de ejecución en escala lineal.

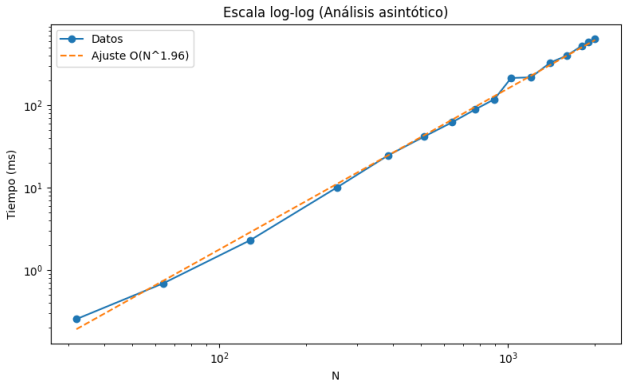

Figura 2: Tiempo de ejecución en escala log-log.

El ajuste realizado sobre los datos experimentales produjo una pendiente aproximada de $k \approx 1{,}96$, lo cual es muy cercano al valor teórico esperado $k = 2$. Esto confirma experimentalmente que el algoritmo de Gale–Shapley presenta un comportamiento cuadrático $O(N^2)$ en el peor caso.

Las pequeñas desviaciones observadas respecto al ajuste teórico pueden atribuirse a factores propios del entorno de ejecución, como administración de memoria, caché del procesador y sobrecarga del sistema operativo. Sin embargo, la tendencia global confirma el crecimiento cuadrático esperado.

5. Conclusiones

Se implementó el algoritmo de Gale–Shapley para resolver el Problema de Emparejamiento Estable y se

evaluó su comportamiento temporal mediante un estudio experimental con distintos tamaños de entrada.

Los resultados obtenidos muestran que el tiempo de ejecución crece de manera aproximadamente cuadrática con respecto a N. El ajuste realizado en escala log-log arrojó una pendiente cercana a 1,96, valor consistente con la complejidad teórica $O(N^2)$ del algoritmo en el peor caso.

Las variaciones observadas entre los datos experimentales y el modelo ideal pueden explicarse por factores externos al algoritmo, como el entorno de ejecución y la administración de recursos del sistema. No obstante, la tendencia general confirma el comportamiento asintótico esperado.

Por lo tanto, el análisis experimental valida empíricamente la complejidad cuadrática del algoritmo de Gale–Shapley, cumpliendo el objetivo planteado en la práctica.

Bibliografía

[1] D. Gale y L. S. Shapley, «College Admissions and the Stability of Marriage,» *The American Mathematical Monthly*, vol. 69, n.º 1, págs. 9-15, 1962.

[2] D. Gusfield y R. W. Irving, *The Stable Marriage Problem: Structure and Algorithms* (Foundations of Computing). Cambridge, MA: MIT Press, 1989.

[3] A. E. Roth y M. A. O. Sotomayor, *Two-Sided Matching: A Study in Game-Theoretic Modeling and Analysis*. Cambridge, United Kingdom: Cambridge University Press, 1990.